



Banco de Dados SQL

Marcel Oliveira

1



Até agora...



Modelo ER

Modelo Relacional

Normalização

SQL



2



Até agora...

- Modelo relacional
- Álgebra e cálculo relacional
 - Base das solicitações que seram feitas em SQL
 - Álgebra
 - Técnica demais
 - Como fazer passo a passo
 - SQL
 - Linguagem declarativa
 - O que fazer? Que propriedades devem ser satisfeitas pelo resultado

3



SQL



- O que significa?
 - Structured Query Language
- Projetada e implementada pela IBM research (Anos 70)
 - Prova de conceito de implementação do modelo relacional
 - Inicialmente SEQUEL (“*Structured English Query Language*”)
 - "síquel" ao invés de "és-kiú-él"
 - Porém, em português "ésse-quê-éle".

4



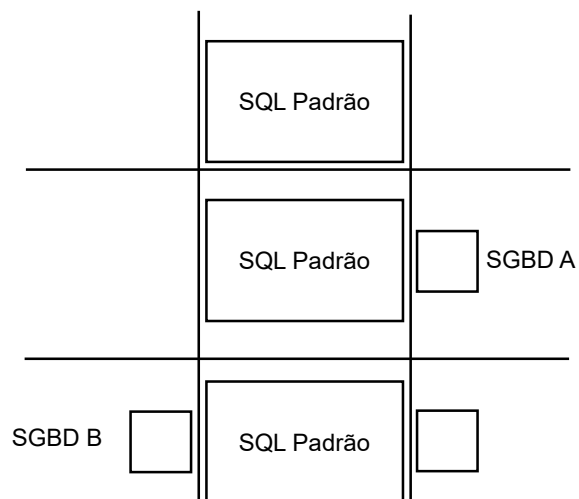
SQL

- Uma das principais razões do sucesso do modelo relacional
- Tornou-se padrão
 - Migração dos modelos hierárquicos e de rede foi estimulada
 - Migração entre SGBDs
- Na prática...

5



SQL



6



SQL

- Versões padronizadas pela ANSI e ISO
 - SQL-86 (ANSI) ou SQL-87 (ISO)
 - SQL-89
 - SQL-92 ou SQL2
 - SQL:1999 ou SQL3
 - SQL:2003
 - SQL:2006
 - SQL:2008
 - SQL:2011
 - SQL:2016
 - SQL:2019
 - SQL:2023

Fonte: <https://en.wikipedia.org/wiki/SQL>

7



SQL

- Vantagens de manter-se no padrão
 - Fazer migrações fáceis de SGBD
 - Uma mesma aplicação pode acessar mais de um BD em SGBDs diferentes
- Vantagens de usar linguagem especializada
 - Simplicidade em algumas operações
 - Otimização

8



SQL

- Baseada fortemente no cálculo relacional de tuplas
- Também utiliza algumas funções da álgebra relacional
- Possui sintaxe mais amigável do que ambos

9



A Linguagem

- DDL: Definições de dados
- DML: Consultas e atualizações
- VDL: Definição de visões
- Outros
 - Segurança
 - Autorizações de acesso
 - Restrições de integridade
 - Controle de transações

10



A Linguagem

- Linguagem central (core language)
 - Suportado pela maioria dos SGBDs
- Pacotes especializados opcionais
 - Mineração de dados, dados temporais, dados multimídia, etc
 - Adquiridos independentemente

11



Mas, para quê SQL?

“Aprender um framework de MOR é muito importante pois facilita principalmente o trabalho de CRUD e consultas básicas (filtros simples, JOINS, etc).

No entanto, há uma série de situações onde é preciso usar SQL diretamente, sendo a principal delas para relatórios que envolvam funções de agregação e agrupamento. É quase regra que quando se trata de um relatório de cálculo ou já com um pouco complexidade usar SQL direto é bem mais indicado.

Já tive casos aqui de termos profissionais que trabalharam somente com frameworks MOR e não sabiam SQL. Qualquer consulta um pouquinho mais complicada ele tentava usar o framework e incorriam em um código com diversos N + 1 Select, anti-perfomático e difícil de entender. Este motivo, inclusive, resultou em seu desligamento por essa insuficiência técnica que ele não conseguiu sanar. Ou seja, é requisito básico :)”



Gleydson Lima
Sócio-Fundador ESig

12



Mas, para quê SQL?

- *Melhoria de desempenho de consultas*
- *Tornar-se um DBA*
- *Business intelligence: extração dos dados e importação em Pentaho, Qlickview, etc será feita usando SQL*

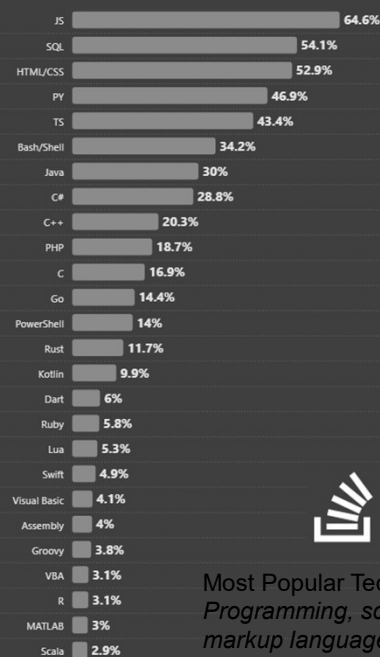


Prof. Itamir Filho
Ex-Diretor de Tecnologia da Informação – IMD
Coordenador da Unidade EMBRAPIL - IMD

13



All Respondents Professional Developers Learning to Code Other Coders



Most Popular Technologies
Programming, scripting, and
markup languages

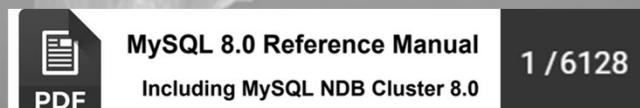
<https://survey.stackoverflow.co/2024/technology#most-popular-technologies-language-prof>

14



Nas Próximas Aulas

- Criação de esquemas e tabelas
- Tipos de dados básicos
- Restrições básicas
 - Integridade referencial e de chave
- Modificações de esquemas, tabelas, e restrições



15



Nas Próximas Aulas

- Consultas básicas
- Consultas complexas
 - Funções agregadas e agrupamento
- Comandos de inserção, remoção e atualização de dados
- Restrições mais gerais sobre o banco (Asserções)
- Triggers
- Visões
- Resumo de outras características de SQL

16



Definição de Dados e Tipos de Dados

- Termos
 - Relação → TABLE
 - Tupla → TUPLE
 - Atributo → COLUMN
- Comando CREATE
 - Esquemas
 - Tabelas
 - Domínios
 - Visões
 - Asserções
 - Triggers

17



Esquemas x Catálogo

- Esquema
 - Identificado por nome
 - Inclui identificador de autorização para indicar o usuário que possui o esquema
 - Descrições dos **elementos**
 - Tabelas
 - Restrições
 - Visões
 - Domínios
 - Outros construtores (autorizações)

18



Esquemas x Catálogo

- Catálogo
 - Uma coleção de esquemas em um ambiente SQL
 - Sempre possui um esquema especial INFORMATION_SCHEMA
 - Informações de todos os esquemas do catálogo e suas descrições
 - Restrições de integridade apenas entre relações de esquemas do mesmo catálogo
 - Esquemas do mesmo catálogo podem compartilhar alguns elementos como definição de domínio

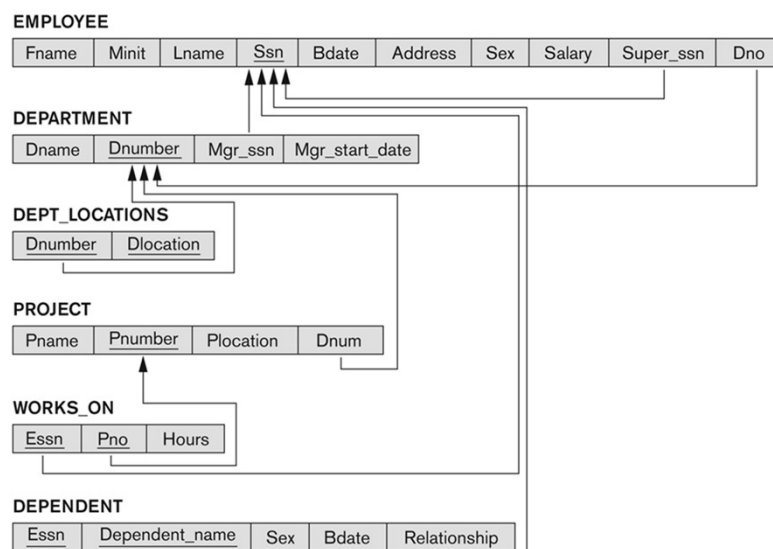
19



Nosso Exemplo

Figure 5.7

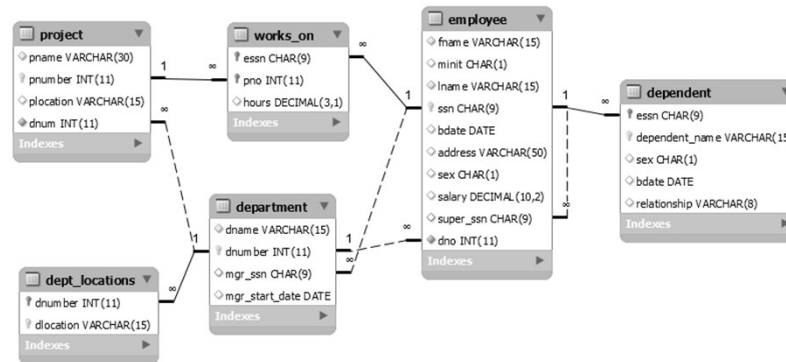
Referential integrity constraints displayed on the COMPANY relational database schema.



20



Nosso Exemplo



21



Criando Esquemas

(create_db.sql)

- Criação
CREATE DATABASE COMPANY;
- Exibição
SHOW DATABASES;
- Remoção
DROP DATABASE COMPANY;
- Seleção
USE COMPANY;

22



Criando Tabelas

(create_tables_1.sql)

- CREATE TABLE
- Cria uma nova tabela
 - Nome
 - Atributos
 - Nome
 - Tipo
 - Restrições do atributo (NOT NULL)
 - Restrições iniciais
 - Chaves
 - Chaves estrangeiras
 - Restrições de integridade
- CREATE TABLE ou ALTER TABLE

23



Comandos em Geral

```
CREATE TABLE  
    COMPANY.EMPLOYEE ...
```

X

```
CREATE TABLE EMPLOYEE...
```

24



Criando Tabelas

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	-----	-------	---------	-----	--------	-----------	-----

employee	
◇	fname VARCHAR(15)
◇	minit CHAR(1)
◇	lname VARCHAR(15)
1	ssn CHAR(9)
◇	bdate DATE
◇	address VARCHAR(50)
◇	sex CHAR(1)
◇	salary DECIMAL(10,2)
◇	super_ssn CHAR(9)
◇	dno INT(11)
Indexes	

```
CREATE TABLE employee (  
    fname VARCHAR(15) NOT NULL,  
    minit CHAR,  
    lname VARCHAR(15) NOT NULL,  
    ssn CHAR(9) NOT NULL,  
    bdate DATE,  
    address VARCHAR(50),  
    sex CHAR,  
    salary DECIMAL(10,2),  
    super_ssn CHAR(9),  
    dno INT NOT NULL,  
    PRIMARY KEY(ssn),  
    FOREIGN KEY(super_ssn) REFERENCES employee(ssn)  
);
```

25



Criando Tabelas

- Outras tabelas
 - department
 - dept_locations
 - project
 - works_on
 - Dependent
- Ainda falta uma chave estrangeira
 - dno de EMPLOYEE

26



Tipos de Dados

- Numéricos
 - INTEGER, INT [4 bytes]
 - SMALLINT [2 bytes]
 - FLOAT [4 bytes]
 - REAL [8 bytes]
 - DECIMAL (i, j) [Varia]
 - i – precisão: total de números decimais
 - j – escala: número de dígitos depois do ponto
 - Vários outros

27



Tipos de Dados

- Caracter/String
 - CHAR (n) : tamanho fixo de n
 - Preenchimento com branco à esquerda
 - Brancos geralmente ignorados em comparações
 - VARCHAR (n) : tamanho variável até n

28



Tipos de Dados

- Bit-string
 - BIT (n) : tamanho fixo de n
 - B'0101'
 - BIT VARYING (n) : tamanho variável até n
 - BLOB : Objetos binários grandes

29



Tipos de Dados

- BOOLEAN
 - true
 - false
 - NULL (UNKNOWN – 3-Values Logic)

30



Tipos de Dados

- TEMPO

- DATE: YYYY-MM-DD
 - DATE '2008-10-16'
- TIME: HH:MM:SS
 - TIME '18:57:54'
- TIME (i)
 - Tempo com mais precisões
- TIME WITH TIME ZONE
 - 6 posições adicionais: +13:00 a -12:59

31



Tipos de Dados

- TEMPO

```
TIMESTAMP  
'2008-10-16 18:57:54 423443'
```

- Domínios

```
CREATE DOMAIN SSN_TYPE as CHAR(9);
```

- Não disponível em várias implementações como MySQL ;-)

32



Restrições

- Integridade referencial e de chave
- Domínios de atributos
- NULLs
- Restrições sobre tuplas individuais

33



Restrições

- NOT NULL
- DEFAULT <valor>

```
(create_tables_2.sql)
CREATE TABLE employee(
...
    dno INT NOT NULL DEFAULT 1,
...
);
```

34



Restrições

- CHECK <condição>

```
CREATE TABLE department(  
...  
    dnumber INT  
        NOT NULL  
        CHECK  
            (dnumber > 0  
             AND dnumber < 21),  
...  
);
```

35



Restrições

- PRIMARY KEY (<atributo>)

```
CREATE TABLE employee(  
...  
    CONSTRAINT EMPPK  
        PRIMARY KEY(ssn),  
...  
);
```

36



Restrições

- FOREIGN KEY (<atributo>)
REFERENCES
<tabela>(<atributo>)

```
CREATE TABLE employee(  
...  
    CONSTRAINT EMPSUPERFK  
        FOREIGN KEY(super_ssn)  
        REFERENCES employee(ssn)  
...  
);
```

37



Restrições

- UNIQUE (<atributo>)
– Chaves secundárias

```
CREATE TABLE department(  
...  
    CONSTRAINT DEPTSK  
    UNIQUE(dname) ,  
...  
);
```

38



Restrições

- Comportamento default à violações: `reject`
- Qualificações
 - `ON DELETE` ou `ON UPDATE`
- Opções
 - `SET NULL`
 - `CASCADE`
 - `SET DEFAULT`

39



Restrições

```
CREATE TABLE employee(  
...  
CONSTRAINT EMPSUPERFK  
  FOREIGN KEY(super_ssn)  
  REFERENCES employee(ssn)  
    ON DELETE SET NULL  
    ON UPDATE CASCADE  
) ;
```

No innoDB, base do MySQL funciona como RESTRICT

40



Restrições

ON UPDATE CASCADE

ON UPDATE SET NULL

"If ON UPDATE CASCADE or ON UPDATE SET NULL recurses to update the same table it has previously updated during the cascade, it acts like RESTRICT."

This means that you cannot use self-referential ON UPDATE CASCADE or ON UPDATE SET NULL operations.

This is to prevent infinite loops resulting from cascaded updates. A self-referential ON DELETE SET NULL, on the other hand, is possible, as is a self-referential ON DELETE CASCADE. Cascading operations may not be nested more than 15 levels deep. "

FONTE: <http://dev.mysql.com/doc/refman/5.0/en/innodb-foreign-key-constraints.html>

41



Restrições

- Outros CHECK
 - Departamento com dept_create_date

```
CREATE TABLE department (  
...  
CONSTRAINT DATE CHECK  
  (dept_create_date <=  
    mgr_start_date)  
);
```

42



Mudanças de Esquema

- DROP
 - Remoção de elementos de esquema
 - Tabelas
 - Domínios
 - Restrições
 - Remoção de esquemas
- Duas opções de comportamento (não em MySQL)
 - CASCADE
 - RESTRICT

43



Mudanças de Esquema

(change_tables.sql)
`DROP TABLE dependent CASCADE;`

- CASCADE: Todas as restrições e visões que referenciam a tabela removida são removidas do esquema
- RESTRICT: Só funciona se tabela não for referenciada no esquema
- `DROP TABLE` remove toda a tabela. Para remover tuplas veremos `DELETE` mais adiante

44



Mudanças de Esquema



45



Mudanças de Esquema

- ALTER
 - Alteração de elementos de esquema
 - Tabelas
 - Adição/Remoção de colunas
 - Mudança de definição de coluna
 - Adição/Remoção de restrições

46



Mudanças de Esquema

```
ALTER TABLE employee  
ADD COLUMN job VARCHAR(12);
```

- Para tuplas já existentes
 - DEFAULT
 - NULL caso contrário

47



Mudanças de Esquema

```
ALTER TABLE employee  
DROP COLUMN address  
CASCADE;
```

- CASCADE: Todas as restrições e visões que referenciam a coluna removida são removidas do esquema
- RESTRICT: Só funciona se coluna não for referenciada no esquema

48



Mudanças de Esquemas

```
ALTER TABLE department
  ALTER COLUMN mgr_ssn
  DROP DEFAULT;
ALTER TABLE department
  ALTER COLUMN mgr_ssn
  SET DEFAULT '33344555';
ALTER TABLE employee
  DROP CONSTRAINT
  EMPSUPERFK;
```

49



Mudanças de Esquema

```
ALTER TABLE employee
  ADD CONSTRAINT
  FOREIGN KEY (dno)
  REFERENCES
  DEPARTMENT (dnumber) ;
```

FIM DA AULA 01

50



Consultas Básicas

- Criando e Populando o BD
(`create_db_and_populate.sql`)
- Modelo
 - `company_model.docx`
- Dados
 - `company_data.xlsx`

51



Consultas Básicas

- `SELECT`
 - Sem relação com o select da álgebra relacional
- Diferenças entre SQL e modelo relacional
 - Tuplas podem ter mesmos valores
 - Tabela é um *bag* de tuplas
 - Restrições para conjuntos são feitas através de chaves e/ou cláusula `DISTINCT`

52



SELECT-FROM-WHERE

```
SELECT <lista de atributos>  
FROM <lista de tabelas>  
WHERE <condição>
```

- Comparadores básicos

- =, <, <=, >, >=, e <>

(basic_queries.sql)

53



SELECT-FROM-WHERE

- Q0: recupere a data de nascimento e endereço do empregado 'John B. Smith'

```
SELECT BDATE, ADDRESS  
FROM EMPLOYEE  
WHERE FNAME='John'  
AND MINIT='B'  
AND LNAME='Smith';
```

- Similar a SELECT-PROJECT

$$\pi_{bdate, address}(\sigma_{fname='John' \wedge minit='B' \wedge lname='Smith'}(EMPLOYEE))$$

- Resultado SQL pode ter tuplas repetidas

54



SELECT-FROM-WHERE

```
SELECT BDATE, ADDRESS
FROM EMPLOYEE
WHERE
    FNAME='John'
    AND MINIT='B'
    AND LNAME='Smith';
```

- Também similar a expressão do cálculo de tuplas

```
{t.bdate, t.address
| EMPLOYEE(t) AND
  t.fname='John' AND
  t.minit='B' AND t.lname='Smith' }
```
- Como se tivéssemos variáveis implícitas de tuplas

55



SELECT-FROM-WHERE

- Q1: Recupere o nome e endereço de todos os empregados do departamento 'Research'

```
SELECT FNAME, LNAME, ADDRESS
FROM EMPLOYEE, DEPARTMENT
WHERE [DNAME='Research'] ← Condição de seleção
      AND [DNUMBER=DNO;] ← Condição de junção
```

- Similar à sequência de operações SELECT-PROJECT-JOIN da álgebra relacional
- Mais de uma condição de junção...

56



SELECT-FROM-WHERE

- Q2: para todo projeto localizado em 'Stafford', liste o número do projeto, o número do departamento que o controla, e o último nome, endereço e data de nascimento do gerente do departamento

```
SELECT PNUMBER, DNUM, LNAME,  
       BDATE, ADDRESS  
FROM PROJECT, DEPARTMENT,  
     EMPLOYEE  
WHERE DNUM=DNUMBER  
      AND MGR_SSN=SSN  
      AND PLOCATION='Stafford';
```

Relaciona o projeto com o departamento

Relaciona o departamento com o seu gerente

57



Nomes de atributos ambíguos, codinome e variáveis de tupla

- Acontece quando
 - temos mesmo nome de atributo para relações diferentes
 - SELECT NAME FROM...
 - Qualificação do atributos
 - EMPLOYEE.fname
 - Temos duas referências (ou mais) para a mesma relação
 - Codinomes

58



Nomes de atributos ambíguos, codinome e variáveis de tupla

- Q8: Para cada empregado, recupera o nome do empregado e o nome de seu supervisor

```
SELECT E.FNAME, E.LNAME,  
       S.FNAME, S.LNAME  
FROM EMPLOYEE AS E, EMPLOYEE AS S  
WHERE E.SUPERSSN=S.SSN;
```

```
SELECT E.FNAME, E.LNAME,  
       S.FNAME, S.LNAME  
FROM EMPLOYEE E, EMPLOYEE S  
WHERE E.SUPERSSN=S.SSN;
```

59



Nomes de atributos ambíguos, codinome e variáveis de tupla

- A prática do uso de codinomes para as relações deveria ser adotada independente da existência de ambiguidade

Q1A:

```
SELECT E.FNAME, E.LNAME,  
       E.ADDRESS  
FROM EMPLOYEE E, DEPARTMENT D  
WHERE D.DNAME='Research'  
      AND E.DNUMBER=D.DNO;
```

60



Nomes de atributos ambíguos, codinome e variáveis de tupla

- Codinomes para atributos

```
EMPLOYEE AS  
E(Fn, Mi, Ln, Ssn, Bd, Sex,  
  Sal, Sssn, Dno)
```

61



Cláusula WHERE e *

- WHERE true pode ser omitida
- Se mais de uma relação faz parte do SELECT temos um produto cartesiano
- Q9: Recupere os SSN dos empregados

```
SELECT SSN FROM EMPLOYEE;
```

- Q10: Todas as combinações de SSN com nome de departamento

```
SELECT SSN, DNAME  
FROM EMPLOYEE, DEPARTMENT;
```

62



Cláusula WHERE e *

- Recuperação de todos os atributos
– *
- Q1C: Recupere todos os atributos dos empregados do departamento 5

```
SELECT * FROM EMPLOYEE WHERE DNO=5;
```
- Q10: Todas os atributos do empregados do departamento 'Research'

```
SELECT *  
FROM EMPLOYEE, DEPARTMENT  
WHERE dname='Research'  
AND dno=dnumber;
```

63



Cláusula WHERE e *

- Cuidado: o esquecimento de condição de junção e seleção pode gerar relações incorretas

Q10A:

```
SELECT *  
FROM EMPLOYEE, DEPARTMENT;
```

64



Tabelas como Conjuntos

- SQL não elimina tuplas repetidas
 - Alto custo: ordenação e eliminação
 - Duplicatas podem ser desejadas
- Eliminação explícita de tuplas
 - DISTINCT
- Q11: Retorne os salários dos empregados

```
SELECT ALL SALARY FROM EMPLOYEE;  
SELECT DISTINCT SALARY  
FROM EMPLOYEE;
```

65



Tabelas como Conjuntos

- Operações sobre conjuntos
 - UNION, EXCEPT, INTERSECT
- Resultados são conjuntos
- Aplicáveis apenas a relações união-compatíveis
 - Mesmos atributos
 - Mesma ordem de atributos

66



Tabelas como Conjuntos

- Q4: Lista de todos os números dos projetos que envolvem um empregado cujo último nome é 'Smith' como trabalhador ou gerente do departamento que controla o projeto

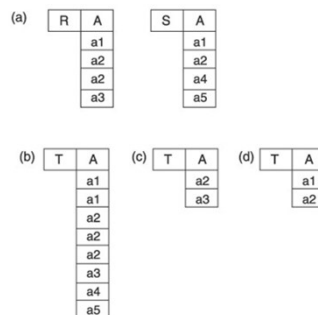
```
(SELECT PNUMBER
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE DNUM=DNUMBER
      AND MGR_SSN=SSN           'Smith' como funcionário
      AND LNAME='Smith')
UNION
(SELECT PNUMBER
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE PNUMBER=PNO
      AND ESSN=SSN              'Smith' como gerente
      AND LNAME='Smith');
```

67



Tabelas como Conjuntos

- Operações sobre bags
 - UNION ALL, EXCEPT ALL, INTERSECT ALL
- Resultados são bags



68



Casamento de Padrões

- LIKE
 - _ : Exatamente um caracter
 - % : Qualquer número de caracteres
 - 'AB_CD\%EF' ESCAPE '\\' = 'AB_CD%EF'
 - \" representa um ' como caracter
- Q12: Todos os empregados que moram em Houston, TX.

```
SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE address LIKE '%Houston,%TX%';
```

69



Casamento de Padrões

- Q12A: Todos os empregados nascidos nos anos 50.

```
SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE bdate LIKE
'__5_____';
```

70



Aritmética em Consultas

- +, -, *, e /
- Q13: Mostre os salários resultantes se todo empregado que trabalha no projeto 'ProductX' receber 10% de aumento

```
SELECT
    fname, lname,
    salary AS old_salary,
    1.1*salary AS increased_salary
FROM EMPLOYEE, WORKS_ON, PROJECT
WHERE
    ssn=essn AND pno=pnumber
    AND pname='ProductX';
```

71



Outras operações em Consultas

- Concatenação de string: ||
- Incremento (+) e decremento (-)
 - DATE, TIME, TIMESTAMP, INTERVAL
- BETWEEN
 - Q14: Recupere os empregados do departamento 5 com salário entre 30000 e 40000.

```
SELECT fname, lname FROM EMPLOYEE
WHERE
    (salary BETWEEN 30000 AND 40000)
    AND dno=5;
```

72



Ordenação de Resultados

- ORDER BY
- ASC e DESC (padrão é ascendente)
- Q15: lista dos empregados e dos projetos que eles trabalham ordenados por departamento, e dentro do departamento, ordenado por último nome, primeiro nome

```
SELECT dname, lname, fname, pname
FROM DEPARTMENT, EMPLOYEE,
      WORKS_ON, PROJECT
WHERE dnumber=dno AND ssn=essn
      AND pno=pnumber
ORDER BY dname, lname, fname;
```

73



Ordenação de Resultados

- ORDER BY
- ASC e DESC (padrão é ascendente)
- Q15: lista dos empregados e dos projetos que eles trabalham ordenados por departamento, e dentro do departamento, ordenado por último nome, primeiro nome

```
SELECT dname, lname, fname, pname
FROM DEPARTMENT, EMPLOYEE,
      WORKS_ON, PROJECT
WHERE dnumber=dno AND ssn=essn
      AND pno=pnumber
ORDER BY dname DESC,
      lname ASC, fname ASC;
```

74



Comparações com NULL e lógica de três valores

- Valores NULL
 - Valor desconhecido
 - Valor indisponível
 - Atributo não se aplica
- SQL não faz distinção
- Em geral, dois valores NULL são diferentes entre si
- Quando NULL está envolvido em uma operação de comparação o resultado é desconhecido
- SQL usa lógica de três valores

75



Operações na Lógica de Três Valores

	AND	TRUE	FALSE	UNKNOWN
TRUE		TRUE	FALSE	UNKNOWN
FALSE		FALSE	FALSE	FALSE
UNKNOWN		UNKNOWN	FALSE	UNKNOWN
	OR	TRUE	FALSE	UNKNOWN
TRUE		TRUE	TRUE	TRUE
FALSE		TRUE	FALSE	UNKNOWN
UNKNOWN		TRUE	UNKNOWN	UNKNOWN
	NOT			
TRUE		FALSE		
FALSE		TRUE		
UNKNOWN		UNKNOWN		

76



Comparações com NULL e lógica de três valores

- Comparações usando = e <> podem não ser apropriadas visto que valores NULL são diferentes entre si
- Tuplas com valores NULL para os atributos de junção não são incluídas
- IS e IS NOT
- Q18: nome dos empregados sem supervisores

```
SELECT fname, lname  
FROM EMPLOYEE  
WHERE super_ssn IS NULL;
```

77



Consultas Entrelaçadas

- Uma consulta entrelaçada pode ser feita dentro da cláusula WHERE de outra consulta

78



Consultas Entrelaçadas

- Q4: Lista de todos os números dos projetos que envolvem um empregado cujo último nome é 'Smith' como trabalhador ou gerente do departamento que controla o projeto

```
(SELECT PNAME
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE DNUM=DNUMBER
      AND MGR_SSN=SSN
      AND LNAME='Smith')
UNION
(SELECT PNAME
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE PNUMBER=PNO
      AND ESSN=SSN
      AND LNAME='Smith');
```

79



Consultas Entrelaçadas

- Q4A: Lista de todos os números dos projetos que envolvem um empregado cujo último nome é 'Smith' como trabalhador ou gerente do departamento que controla o projeto

```
SELECT DISTINCT pnumber
FROM PROJECT
WHERE pnumber IN
  (SELECT pnumber
   FROM PROJECT, DEPARTMENT, EMPLOYEE
   WHERE DNUM=DNUMBER
         AND MGR_SSN=SSN
         AND LNAME='Smith')
OR pnumber IN
  (SELECT pno
   FROM WORKS_ON, EMPLOYEE
   WHERE ESSN=SSN
         AND LNAME='Smith');
```

'Smith' como funcionário

'Smith' como gerente

80



Consultas Entrelaçadas

- Tuplas de valores em comparações
- QX1: Recupere os `ssns` de todos os empregados que trabalham a mesma combinação (projeto, horas) em algum projeto que o funcionário '123456789' trabalha

```
SELECT DISTINCT essn
FROM WORKS_ON
WHERE (pno, hours)
IN (SELECT pno, hours
    FROM WORKS_ON
    WHERE essn='123456789');
```

81



Consultas Entrelaçadas

- Outros operadores
 - `v IN S`
 - `v = ANY S`
 - Operadores `>`, `<`, `<=`, `>=`, `<>`
 - `V op ALL S (v > ALL S)`
- QX2: Recupere os nomes dos empregados cujo salário é maior do que os salários de todos os empregados do departamento 5

```
SELECT lname, fname
FROM EMPLOYEE
WHERE salary > ALL (SELECT salary
    FROM EMPLOYEE
    WHERE dno='5');
```

82



Consultas Entrelaçadas

- Podemos ter vários níveis de entrelaçamento
- Ambiguidade
 - Referências a atributos sem qualificação se referem a relação declaradas no nível mais interno
 - Q4A

```
SELECT DISTINCT pnumber
FROM PROJECT
WHERE pnumber IN
    (SELECT pnumber
     FROM PROJECT, DEPARTMENT, EMPLOYEE
     WHERE DNUM=DNUMBER
           AND MGR_SSN=SSN
           AND LNAME='Smith')
```

83



Consultas Entrelaçadas

- Q16: recupere o nome de cada empregado que tem um dependente com o mesmo primeiro nome e sexo que o empregado

```
SELECT E.FNAME, E.LNAME
FROM EMPLOYEE AS E
WHERE E.SSN IN
    (SELECT essn
     FROM DEPENDENT
     WHERE E.ssn = essn
           AND E.FNAME=DEPENDENT_NAME
           AND E.sex=sex);
```

84



Consultas Entrelaçadas Correlatas

- Se uma condição na cláusula `WHERE` de uma consulta entrelaçada referencia um atributo de uma relação declarada na consulta mais externa, as duas consultas são correlatas
 - O resultado de uma consulta entrelaçada correlata é diferente para cada tupla (ou combinação de tuplas) da relação mais externa

85



Consultas Entrelaçadas Correlatas

```
SELECT E.FNAME, E.LNAME
FROM EMPLOYEE AS E
WHERE E.SSN IN
    (SELECT essn
     FROM DEPENDENT
     WHERE
         E.ssn = essn
         AND E.FNAME=DEPENDENT_NAME
         AND E.sex=sex);
```

- Para cada tupla de `EMPLOYEE`, calcule a consulta entrelaçada. Se o `ssn` da tupla de empregado estiver no resultado, selecione a tupla

86



Consultas Entrelaçadas Correlatas

- Podemos reescrever algumas consultas entrelaçadas que utilizem = ou IN

```
SELECT E.FNAME, E.LNAME
FROM EMPLOYEE AS E,
      DEPENDENT AS D
WHERE E.SSN=D.essn
      AND E.FNAME=D.DEPENDENT_NAME
      AND E.sex=D.sex;
```

87



EXISTS

- Retorna TRUE se existe a resposta de uma consulta entrelaçada correlata não é vazia

```
SELECT E.FNAME, E.LNAME
FROM EMPLOYEE AS E
WHERE EXISTS
      (SELECT *
      FROM DEPENDENT
      WHERE E.SSN=essn
            AND E.FNAME=DEPENDENT_NAME
            AND E.sex=sex);
```

88



NOT EXISTS

- O contrário de EXISTS
- Q6: Recupere os nomes dos funcionários que não têm dependentes

```
SELECT FNAME, LNAME  
FROM EMPLOYEE  
WHERE NOT EXISTS
```

```
(SELECT *  
FROM DEPENDENT  
WHERE SSN=ESSN) ;
```

Seleciona tuplas relacionadas
para cada um dos
funcionários

89



EXISTS

- Q7: Recupere os nomes dos gerentes que têm pelo menos um dependente

```
SELECT FNAME, LNAME  
FROM EMPLOYEE  
WHERE
```

```
EXISTS  
(SELECT * FROM DEPENDENT  
WHERE SSN=ESSN)
```

AND

```
EXISTS  
(SELECT * FROM DEPARTMENT  
WHERE SSN=MGR_SSN) ;
```

Tem pelo menos um
dependente

Gerenciam pelo
menos
um departamento

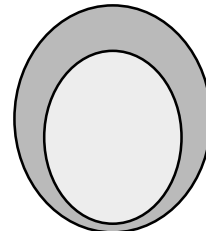
90



EXISTS

- Q3: Recupere o nome de cada empregado que trabalha em todos os projetos controlados pelo departamento '5'

```
SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE
    ( (SELECT pno
      FROM WORKS_ON
      WHERE ssn=essn)
      CONTAINS
      (SELECT pnumber
       FROM PROJECT
       WHERE dnum='5') ) ;
```



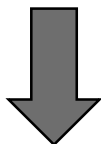
Não é parte de SQL
Precisamos de outra
técnica

91



EXISTS

Empregado **E** trabalha em todos os
projetos controlados pelo departamento '5'



$$\forall x. P(x) \Leftrightarrow \neg \exists x. \neg P(x)$$

Não existe um projeto controlado pelo
departamento '5' que o funcionário **E**
não trabalhe

92



EXISTS

- Q3: Recupere o nome de cada empregado que trabalha em todos os projetos controlados pelo departamento '5'

```
SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE
  NOT EXISTS
  ( (SELECT * FROM WORKS_ON W1
    WHERE (W1.pno IN
      (SELECT pnumber
       FROM PROJECT
       WHERE dnum=5))
    AND
    NOT EXISTS (SELECT *
      FROM WORKS_ON W2
      WHERE W2.essn=ssn
      AND W1.pno=W2.pno)
  ) );
```

Projeto do Departamento '5'

Funcionário não trabalha em algum projeto do departamento '5'

Não existe um projeto no departamento '5' que o funcionário não trabalhe

93



UNIQUE

- UNIQUE (C)
 - Retorna TRUE se não existirem tuplas repetidas no resultado da consulta C
 - Conjunto ou Bag?

94



Conjuntos Explícitos

Q17: Recupere os SSN dos empregados que trabalham nos projetos 1, 2, ou 3

```
SELECT DISTINCT ESSN
FROM WORKS_ON
WHERE PNO IN (1, 2, 3);
```

95



Renomeação

- Q8A: recupere o último nome dos empregados e seus supervisores

```
SELECT
    E.lname AS Employee_name,
    S.lname AS Supervisor_name
FROM
    EMPLOYEE AS E,
    EMPLOYEE AS S
WHERE E.super_ssn=S.ssn;
```

96



Junção de Tabelas

- Podemos definir junção de tabelas na cláusula `FROM`
- É uma relação, mas resultante de uma junção
- Diferentes junções
 - `JOIN`
 - `NATURAL JOIN`
 - `LEFT OUTER JOIN`
 - `RIGHT OUTER JOIN`
 - `CROSS JOIN`
 - Etc
- Facilita o entendimento da consulta ao retirar junções da cláusula `WHERE`

97



Junção de Tabelas

- Q1: Recupere o nome e endereço de todos os empregados do departamento 'Research'

```
SELECT FNAME, LNAME, ADDRESS  
FROM EMPLOYEE, DEPARTMENT  
WHERE DNAME='Research' AND DNUMBER=DNO;
```

Q1A:

```
SELECT fname, lname, address  
FROM EMPLOYEE JOIN DEPARTMENT ON  
    dno=dnumber  
WHERE dname='Research';
```

98



Analizado Eficiência da Consulta (MySQL)

- Q1A Explain
 - EXPLAIN
- Q3 STATS
 - *Query Stats*



– *Execution Plan*



99



Junção de Tabelas

- NATURAL JOIN
- Q1: Recupere o nome e endereço de todos os empregados do departamento 'Research'

Q1B:

```
SELECT fname, lname, address
FROM
    EMPLOYEE NATURAL JOIN
    (SELECT dname,
            dnumber AS dno,
            mgr_ssn AS mssn,
            mgr_start_date AS msdate
     FROM DEPARTMENT) AS DEPTO
WHERE dname='Research';
```

100



Junção de Tabelas

- NATURAL JOIN
- Caso não existam colunas com mesmo nome, o resultado será um produto cartesiano

```
SELECT dno, dnumber  
FROM EMPLOYEE NATURAL JOIN DEPARTMENT;
```

101



Junção de Tabelas

- Junção padrão é INNER JOIN
 - Apenas tuplas que casam são incluídas
- OUTER JOIN
 - Inclui todas as tuplas
- Q8: Recupere o último e primeiro nome do funcionário e de seu supervisor (NULL se inexistente)

```
Q8A:  
SELECT E.lname AS Employee_name,  
       S.lname AS Supervisor_name  
FROM EMPLOYEE AS E, EMPLOYEE AS S  
WHERE E.super_ssn=s.ssn;
```

```
Q8B:  
SELECT E.lname AS Employee_name,  
       S.lname AS Supervisor_name  
FROM (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S  
      ON E.super_ssn=S.ssn);
```

102



Junção Entrelaçada

- Q2: para todo projeto localizado em 'Stafford', liste o número do projeto, o número do departamento que o controla, e o último nome, endereço e data de nascimento do gerente do departamento

```
SELECT PNUMBER, DNUM, LNAME, BDATE,
       ADDRESS
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE DNUM=DNUMBER
      AND MGR_SSN=SSN
      AND PLOCATION='Stafford';
```

103



Junção Entrelaçada

- Q2A: para todo projeto localizado em 'Stafford', liste o número do projeto, o número do departamento que o controla, e o último nome, endereço e data de nascimento do gerente do departamento

```
SELECT PNUMBER, DNUM, LNAME, BDATE,
       ADDRESS
FROM ((PROJECT JOIN DEPARTMENT ON
      DNUM=DNUMBER) JOIN
      EMPLOYEE ON MGR_SSN=SSN)
WHERE PLOCATION='Stafford';
```

FIM DA AULA 02

104



Funções Agregadas

- COUNT, SUM, MAX, MIN, AVG
- Podem ser usados na cláusula SELECT ou HAVING
- MAX e MIN exigem apenas uma ordem total entre os elementos do tipo

105



Funções Agregadas Exemplos

- Q19: Retorne a soma, o salário máximo, o salário mínimo, e o salário médio do salário de todos os empregados

```
SELECT SUM(salary),  
        MAX(salary),  
        MIN(salary),  
        AVG(salary)  
FROM EMPLOYEE;
```

106



Funções Agregadas Exemplos

- Q20: Q19 mas apenas para os funcionários do departamento 'Research'

```
SELECT SUM(salary), MAX(salary),  
       MIN(salary), AVG(salary)  
FROM (EMPLOYEE JOIN DEPARTMENT  
      ON dno=dnumber)  
WHERE dname='Research';
```

107



Funções Agregadas Exemplos

- Q21 e 22: Número total de empregados da companhia (Q21) e do departamento 'Research' (Q22)

Q21:

```
SELECT COUNT(*) FROM EMPLOYEE;
```

Q22:

```
SELECT COUNT(*)  
FROM EMPLOYEE, DEPARTMENT  
WHERE dno=dnumber AND dname='Research';
```

108



Funções Agregadas Exemplos

- Q23: Conte o número de salários diferentes na empresa

```
SELECT COUNT(DISTINCT salary)
FROM EMPLOYEE;
```

109



Funções Agregadas Exemplos

- COUNT(salary) vs COUNT(DISTINCT salary)?
- Em geral, tuplas com salário NULL são descartadas e não contadas
- Funções agregadas também podem ser usadas em condições de seleção

110



Funções Agregadas Exemplos

- Q5: Nome de todos os empregados que têm mais de um dependente

```
SELECT lname, fname  
FROM EMPLOYEE  
WHERE  
(SELECT COUNT(*)  
FROM DEPENDENT  
WHERE ssn=essn) > 1;
```

111



Agrupamento

- Permite a aplicação de funções agregadas a subgrupos de tuplas agrupados de acordo com os valores de alguns atributos
 - Média de salários de cada departamento
- Definição de partições (grupos)
 - GROUP BY
 - Faz parte da cláusula SELECT

112



Agrupamento

- Q24: Para cada departamento, recupere o código do departamento, o número de empregados, e o salário médio deles

```
SELECT dno,  
       COUNT(*) AS num_employee,  
       AVG(salary) as avg_salary  
FROM EMPLOYEE  
GROUP BY dno;
```

113



Agrupamento

PNOME	MINICIAL	LNOME	SSN	• • •	SALARIO	SUPERSSN	DNO
John	B	Smith	123456789	• • •	30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453		25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	null	1

DNO	COUNT (*)	AVG (SALARIO)
5	4	33250
4	3	31000
1	1	55000

Resultado da Q24

114



Agrupamento

- Um grupo novo é criado para tuplas com valor `NULL` para o atributo de agrupamento
- Podemos ter uma condição de junção junto com um agrupamento

115



Agrupamento

- Q25: Para cada projeto, recupere o número do projeto, o nome do projeto, e o número de empregados que trabalham naquele projeto

```
SELECT PNUMBER, PNAME, COUNT(*)  
FROM PROJECT, WORKS_ON  
WHERE PNUMBER=PNO  
GROUP BY PNUMBER, PNAME;
```

- Agrupamento e contagem é feito apenas após a junção

116



Agrupamento

- E se quisermos incluir apenas projetos com mais de dois empregados?
- Cláusula HAVING

Q26:

```
SELECT PNUMBER, PNAME, COUNT(*)  
FROM PROJECT, WORKS_ON  
WHERE PNUMBER=PNO  
GROUP BY PNUMBER, PNAME  
HAVING COUNT(*) > 2;
```

117



Agrupamento

PNAME	PNUMERO		ESSN	PNO	HORAS
ProdutoX	1		123456789	1	32,5
ProdutoX	1		453453453	1	20,0
ProdutoY	2		123456789	2	7,5
ProdutoY	2		453453453	2	20,0
ProdutoY	2		333445555	2	10,0
ProdutoZ	3		666884444	3	40,0
ProdutoZ	3		333445555	3	10,0
Automacao	10	...	333445555	10	10,0
Automacao	10		999887777	10	10,0
Automacao	10		987987987	10	35,0
Reorganizacao	20		333445555	20	10,0
Reorganizacao	20		987654321	20	15,0
Reorganizacao	20		888665555	20	null
NovosBeneficios	30		987987987	30	5,0
NovosBeneficios	30		987654321	30	20,0
NovosBeneficios	30		999887777	30	30,0

Esses grupos não são
selecionados por HAVING,
condição da Q26

118



Agrupamento Mais exemplos

- Restringindo as tuplas a serem contadas...
- Q27: Para cada projeto, recupere o número do projeto, o nome do projeto, e o número de empregados do departamento '5' que trabalham no projeto

```
SELECT PNUMBER, PNAME, COUNT(*)  
FROM PROJECT, WORKS_ON, EMPLOYEE  
WHERE PNUMBER=PNO  
      AND SSN=ESSN AND DNO=5  
GROUP BY PNUMBER, PNAME;
```

119



Agrupamento

- Permite uma melhor maneira de fazer a divisão da álgebra relacional

T1	A	B
	a1	b1
	a1	b2
	a1	b3
	a2	b1
	a2	b3
	a3	b2
	a3	b3
	a3	b4
	a4	b1

T2	B
	b2
	b3

T3	A
	a1
	a3

pode ser substituído por *
se B não se repete em T1

```
SELECT A  
FROM T1  
WHERE B IN ( SELECT B FROM T2 )  
GROUP BY A  
HAVING COUNT(DISTINCT B) =  
      (SELECT COUNT(*) FROM T2);
```

A Simpler (and Better) SQL Approach to Relational Division. Victor M. Matos. *Journal of Information Systems Education*, Vol. 13(2)

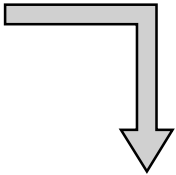
120



EXISTS

- Q3Op: Recupere o nome de cada empregado que trabalha em todos os projetos controlados pelo departamento '5'

```
SELECT fname, lname
FROM (works_on JOIN employee ON essn=ssn)
WHERE pno IN
    (SELECT pnumber
     FROM project
     WHERE dnum=5)
GROUP BY essn
HAVING COUNT(DISTINCT pno) =
    (SELECT COUNT(*)
     FROM project
     WHERE dnum=5);
```



Neste caso
DISTINCT pno
pode ser substituído por *

121



Agrupamento

- Devemos tomar cuidado quando queremos satisfazer duas condições diferentes (uma na cláusula SELECT e outra na HAVING)

122



Agrupamento

- Q28: Retorne o número total, em cada departamento, de empregados com o salário maior que 40000, mas apenas para departamentos com mais de 5 empregados

Apenas departamentos que têm mais de 5 empregados que ganham mais de 40000 serão selecionados

```
SELECT DNAME, COUNT(*)
FROM DEPARTMENT, EMPLOYEE
WHERE dnumber=dno AND salary>40000
GROUP BY dname
HAVING COUNT(*) > 5;
```

WHERE é executado antes para selecionar tuplas;
HAVING é executado depois...

123



Agrupamento

- Q28: Retorne o número total, em cada departamento, de empregados com o salário maior que 40000, mas apenas para departamentos com mais de 5 empregados

```
SELECT DNAME, COUNT(*)
FROM DEPARTMENT, EMPLOYEE
WHERE dnumber=dno AND salary>40000
AND dno IN (SELECT dno
FROM EMPLOYEE
GROUP BY dno
HAVING COUNT(*) > 5)
GROUP BY dnumber;
```

Departamento com mais de 5 funcionários

124



Claúsulas Recursivas (Fecho Transitivo)

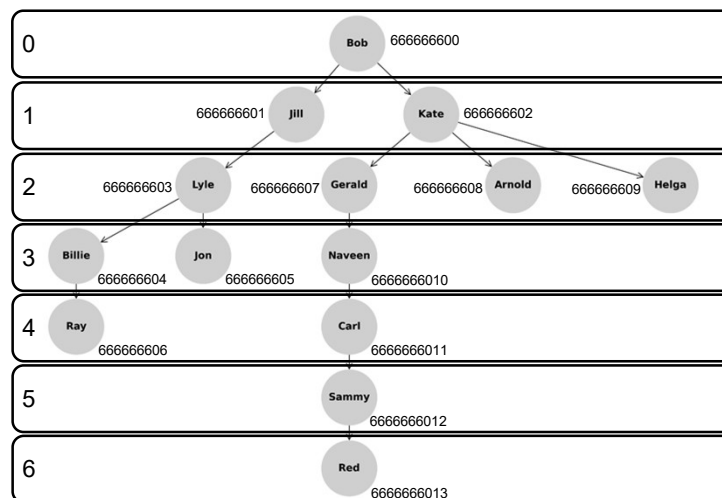
- Q29: Retorne todos os supervisionados por 'Bob' (666666600), não apenas os supervisionados diretos, mas também os supervisionados dos supervisionados, e assim por diante até o fim da hierarquia.

125



Claúsulas Recursivas (Fecho Transitivo)

Árvore de supervisionados de Bob



126



Claúsulas Recursivas (Fecho Transitivo)

- Estrutura geral

```
WITH RECURSIVE table_name (column1, ..., columnn)
AS (
    -- Parte 1: base case (starting point)
    SELECT ...

    UNION ALL

    -- Parte 2: recursive step (add new results)
    SELECT ... // references table_name
)

SELECT * FROM table_name;
```

Bob

todos que têm como
super_ssn alguém
já encontrado.

O banco repete sozinho o passo
recursivo até não achar mais
resultados novos.

127



Claúsulas Recursivas (Fecho Transitivo)

- Q29 (Supervisionados de 'Bob')

```
WITH RECURSIVE hierarchy (ssn, fname, super_ssn, depth)
AS ( -- Base case: Bob
    SELECT e.ssn, e.fname, e.super_ssn, 0 AS depth
    FROM employee e
    WHERE e.fname = 'Bob'

    UNION ALL

    -- Recursive step: everyone that is already
    -- supervised by someone in the set
    SELECT e2.ssn, e2.fname, e2.super_ssn, s.depth + 1
    FROM employee e2
    JOIN hierarchy s ON e2.super_ssn = s.ssn )

SELECT ssn, fname, super_ssn, depth
FROM hierarchy
ORDER BY depth, fname;
```

128



Resumo de Consultas SQL

- Estrutura geral da consulta

SELECT

<lista de atributos e funções>

FROM <lista de tabelas>

[**WHERE** <condição>]

[**GROUP BY**

<atributos de agrupamento>]

[**HAVING**

<condição de agrupamento>]

[**ORDER BY**

<lista de atributos>]

129



Resumo de Consultas SQL

- Ordem conceitual de valoração da consulta

SELECT

<lista de atributos e funções>

FROM <lista de tabelas>

[**WHERE** <condição>]

[**GROUP BY**

<atributos de agrupamento>]

[**HAVING**

<condição de agrupamento>]

[**ORDER BY**

<lista de atributos>]

130



Resumo de Consultas SQL

- Ordem conceitual de valoração da consulta

SELECT

<lista de atributos e funções>

FROM <lista de tabelas>

WHERE <condição>

- Para cada combinação de tuplas das tabelas do **FROM**, verifique se a condição do **WHERE** é **TRUE**; neste caso, selecione os atributos do **SELECT** e coloque-os na resposta da consulta
- Cada SGBD implementa isso de maneira mais otimizada (Capítulos 19 e 20)

131



Considerações

- Existem várias maneiras de se fazer uma mesma consulta
 - Ex: Condições de junção na cláusula **WHERE**, ou junção de relação na cláusula **FROM**, ou consultadas entrelaçadas e operador **IN**
 - Usuário pode escolher que técnica utilizar
 - Preferível utilizar o menor número possível de entrelaçamento
 - Várias possibilidades pode confundir o usuário

132



Considerações

- Existem várias maneiras de se fazer uma mesma consulta
 - Usuário pode utilizar técnica menos eficiente
 - O SGBD deveria tornar isso transparente ao usuário
 - Na prática é bom saber os atalhos do SGBD

133



INSERT

- Adiciona tuplas a uma relação
- Especificamos nome da relação e lista de valores da tupla (ordem deve ser respeitada)

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

(modify_queries.sql)

U1:

INSERT INTO EMPLOYEE

```
VALUES ('Richard','K','Marini',  
        '653298653', '1952-12-30',  
        '98 Oak Forest, Katy, TX',  
        'M', 37000, '987654321',  
        '4');
```

134



INSERT

- Especificamos explicitamente que atributos estão na lista de valores passada
 - Valores `NULL` para os não presentes

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	-----	-------	---------	-----	--------	-----------	-----

U1A:

```
INSERT INTO
  EMPLOYEE (fname, lname, dno, ssn)
VALUES
  ('Richard', 'Marini', '4', '653298653');
```

135



INSERT

- SGBD deveriam garantir todas as restrições de integridade especificadas na DDL
 - Ex: Integridade referencial
- Alguns SGBD não o fazem por eficiência

U2:

```
INSERT INTO EMPLOYEE (fname, lname, ssn, dno)
VALUES
  ('Robert', 'Hatcher', '980760540', '2');
```

U2A:

```
INSERT INTO EMPLOYEE (fname, lname, dno)
VALUES ('Robert', 'Hatcher', '5'); (ssn=NULL)
```

136



INSERT

- Podemos inserir várias tuplas ao mesmo tempo em uma tabela
- U3A e U3B: Tabela contendo nome, número de empregados e salário total dos empregados de cada departamento

```
CREATE TABLE DEPTS_INFO
  (dept_name VARCHAR(15),
   no_of_emps INTEGER, total_sal INTEGER);

INSERT INTO
  DEPTS_INFO(dept_name, no_of_emps, total_sal)
  (SELECT  dname, COUNT(*), SUM(salary)
   FROM (DEPARTMENT JOIN EMPLOYEE ON
         dnumber=dno) GROUP BY DNAME);
```

137



INSERT

- Alterações em DEPARTMENT ou EMPLOYEE não alteram DEPTS_INFO
- DEPTS_INFO pode ficar desatualizada
- Visões devem ser utilizadas para que isso aconteça

138



DELETE

- Remove tuplas (zero, uma, ou mais) de uma relação
- Possui uma cláusula `WHERE` similar ao `SELECT`
- Tuplas são removidas explicitamente de apenas uma tabela

139



DELETE

- Remoções podem ser propagadas para outras relações de ações de propagação foram definidas nas restrições de integridade referencial da DDL

```
CREATE TABLE employee (  
    ...  
    FOREIGN KEY (super_ssn)  
    REFERENCES employee (ssn)  
    ON DELETE <SET NULL | CASCADE>  
    ON UPDATE CASCADE  
);
```

140



DELETE

- Sem a cláusula `WHERE`,
removemos todas as tuplas
 - Tabela é mantida no BD
 - `DROP TABLE <nome>`

U4A: Tenta remover 'Brown', mas ele não existe

```
DELETE FROM EMPLOYEE  
WHERE lname='Brown';
```

141



DELETE

U4B: Remove 'John Smith', e
consequentemente altera as tabelas
`DEPENDENT` e `WORKS_ON`

```
DELETE FROM EMPLOYEE  
WHERE SSN='123456789';
```

142



DELETE

U4C: Remove todos os funcionários do departamento de pesquisa, e consequentemente altera as tabelas DEPENDENT, WORKS_ON, DEPARTMENT, e EMPLOYEE

```
DELETE FROM EMPLOYEE
WHERE dno IN
    (SELECT DNUMBER
     FROM DEPARTMENT
     WHERE DNAME='Research');
```

143



DELETE

U4D: Remove todos os funcionários e consequentemente altera as tabelas DEPENDENT, WORKS_ON, DEPARTMENT, e EMPLOYEE

```
DELETE FROM EMPLOYEE;
```

144



UPDATE

- Modifica valores dos atributos de uma ou mais tuplas de uma relação
- Alteram apenas uma relação
- A cláusula WHERE seleciona que tuplas devem ter os valores alterados
- Alteração nas chaves primárias podem ser propagadas para as chaves estrangeiras de outras relações
- Uma cláusula adicional, SET, estabelece o novo valor dos atributos

145



UPDATE

- **U5: Altere a localização e o departamento que controla o projeto '10' para 'Bellaire' e '5'.**

```
UPDATE PROJECT
SET PLOCATION = 'Bellaire', DNUM = 5
WHERE PNUMBER=10;
```

146



UPDATE

- Várias tuplas podem ser alteradas ao mesmo tempo
- U6: Aumente o salário dos funcionários do departamento 'Research' em 10%.

```
UPDATE EMPLOYEE
SET salary = salary*1.1
WHERE dno IN
    (SELECT dnumber
     FROM DEPARTMENT
     WHERE dname='Research');
```

147



Asserções

- Restrições gerais podem ser descritas usando asserções

```
CREATE_ASSERTION <nome>
CHECK <condição>
```

- Não suportada por MySQL

- Exemplo

- Salário do empregado deve ser menor que o salário do gerente do departamento que ele trabalha

```
CREAT ASSERTION SALARY_CONSTRAINT
CHECK (NOT EXISTS
    (SELECT *
     FROM EMPLOYEE E,
          EMPLOYEE M, DEPARTMENT D
     WHERE E.SALARY > M.SALARY
           AND E.DNO=D.NUMBER
           AND D.MGRSSN=M.SSN) );
```

148



Assertões

- CHECK **vs** CREATE ASSERTION
 - CHECK é verificada apenas quando tuplas são inseridas ou alteradas
 - Por eficiência, recomenda-se seu uso se tivermos certeza que a restrição só pode ser quebrada por essas operações

149



Gatilhos

- CREATE ASSERTION **aborta** a operação caso a condição seja violada
- Gatilhos permitem especificar o tipo de ação a tomar quando um evento ocorre e alguma condição é satisfeita
 - Informar usuário da violação
 - Executar procedimentos armazenados
 - Executar atualizações

150



Gatilhos

- Especificamos um evento, uma condição, e uma ação

```
CREATE TRIGGER
  <nome do gatilho>
  <instante do gatilho>
  <evento do gatilho>
ON <nome da tabela>
[FOR EACH ROW]
[WHEN <condição>]
  <comando do gatilho>
```

BEFORE ou AFTER

INSERT, DELETE, ou UPDATE

151



Gatilhos

- Não podemos ter dois triggers para o mesmo instante e evento em uma mesma tabela
- Comando
 - BEGIN ... END
 - OLD.atributo e NEW.atributo
 - Sintaxe varia bastante entre os SGBDs
 - Em MySQL...

152



Gatilhos

```
(assertion_views.sql)
DELIMITER $$
CREATE TRIGGER SALARY_T
    BEFORE INSERT ON EMPLOYEE
    FOR EACH ROW
    BEGIN
        IF NEW.salary >
            (SELECT salary
             FROM EMPLOYEE
             WHERE ssn=NEW.super_ssn)
        THEN SET NEW.salary = 0;
        END IF;
    END$$
DELIMITER ;
```

153



Visões

- Tabelas simples derivadas de outras tabelas (ou visões)
- Consultas podem ser feitas naturalmente sobre visões
- Tabela virtual
 - Não necessariamente existe fisicamente
 - Limita as atualizações
- SQL

```
CREATE VIEW <nome da visão>
AS <consulta SQL>
```

154



Visões

WORKS_ON_NEW

fname	lname	pname	hours
-------	-------	-------	-------

```
CREATE VIEW WORKS_ON_NEW AS
  SELECT FNAME, LNAME, PNAME, HOURS
    FROM EMPLOYEE, PROJECT, WORKS_ON
   WHERE SSN=ESSN AND PNO=PNUMBER;

SELECT * FROM WORKS_ON_NEW;
```

155



Visões

DEPT_INFO

dept_name	no_of_emps	total_sal
-----------	------------	-----------

```
CREATE VIEW
  DEPT_INFO(dept_name, no_of_emps,
            total_sal)
AS SELECT dname, COUNT(*), SUM(salary)
   FROM DEPARTMENT, EMPLOYEE
  WHERE dnumber=dno
  GROUP BY dname;

SELECT * FROM DEPT_INFO;
```

156



Visões

- Também podem ser usadas como mecanismo de acesso e segurança
- Sempre estão atualizadas
 - Alterações nas tabelas implicam em alterações nas visões
 - Visões são executadas não na criação, mas nas consultas

157



Implementando Visões

- Duas abordagens principais
 - Modificação de consulta
 - Materialização da visão

158



Implementando Visões

- Modificação de consulta
 - Altera a consulta da visão em uma consulta nas tabelas base

```
CREATE VIEW WORKS_ON_NEW AS
  SELECT FNAME, LNAME, PNAME, HOURS
    FROM EMPLOYEE, PROJECT, WORKS_ON
   WHERE SSN=ESSN AND PNO=PNUMBER
   GROUP BY PNAME;
SELECT FNAME, LNAME FROM WORKS_ON_NEW WHERE
  PNAME='ProductX';
```



```
SELECT FNAME, LNAME
  FROM EMPLOYEE, PROJECT, WORKS_ON
 WHERE SSN=ESSN AND PNO=PNUMBER AND NAME='ProductX'
 GROUP BY PNAME;
```

159



Implementando Visões

- Modificação de consulta
 - Ineficiente para consultas complexas
 - Ineficiente se várias consultas são feitas à visão

160



Implementando Visões

- Materialização da visão
 - Cria uma tabela temporária na primeira consulta e a mantém
 - Precisa de mecanismos eficientes de atualização das tabelas materializadas
 - Técnicas de atualização incremental
 - Mantém as tabelas enquanto elas estão sendo consultadas
 - Não existente em MySQL

161



Atualizando Visões

- Complicado e Ambíguo
 - Não é o caso para visões sobre uma tabela sem funções agregadas
- Visões envolvendo junções podem ser mapeadas em várias atualizações de tabelas base. Cuidado!!!

162



Atualizando Visões

```
UPDATE WORKS_ON_NEW  
SET pname='ProductY'  
WHERE lname='Smith' AND fname='John'  
      AND pname='ProductX'
```



```
UPDATE WORKS_ON  
SET pno= (SELECT pnumber  
          FROM PROJECT  
          WHERE pname='ProductY')  
WHERE essn IN (SELECT ssn  
              FROM EMPLOYEE  
              WHERE lname='Smith'  
                AND fname='John')  
  
AND  
pno = (SELECT pnumber  
      FROM PROJECT  
      WHERE pname='ProductX');
```



163



Atualizando Visões

```
UPDATE WORKS_ON_NEW  
SET pname='ProductY'  
WHERE lname='Smith' AND fname='John'  
      AND pname='ProductX'
```



```
UPDATE PROJECT  
SET pname = 'ProductY'  
WHERE pname='ProductX';
```



164



Atualizando Visões

- Visões definidas usando grupos e funções agregadas não são atualizáveis
- Visões definidas sobre várias tabelas usando junção geralmente não são atualizáveis
- **WITH CHECK OPTION**: deve ser adicionado à definição da visão se ela for ser atualizada

165



Outras Características de SQL

- Linguagens de programação e SQL
 - SQL embutido no código
 - ODBC
 - SQL/PSM
- SGBDs têm seus próprios comandos para especificar parâmetros de projeto físico de BD (SDL)
 - Ex: Criação de índices

166



Índices

{ score: 25, ... }	{ score: 56, ... }	{ score: 45, ... }	{ score: 75, ... }	{ score: 5, ... }	{ score: 40, ... }	{ score: 18, ... }	...	{ score: 30, ... }

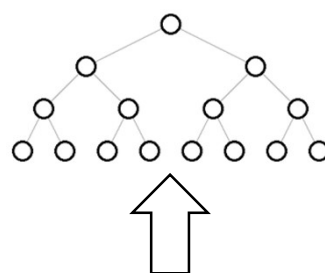
users

- O que acontece se formos procurar um usuário com `score = 30`?

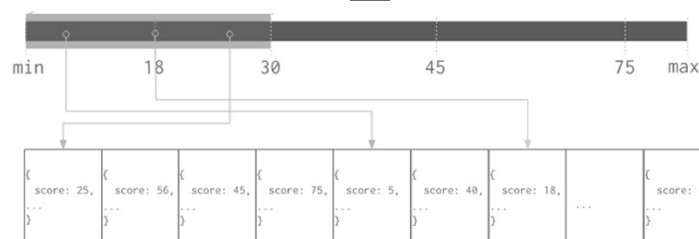
167



Índices



Btree
Busca em $\log(n)$



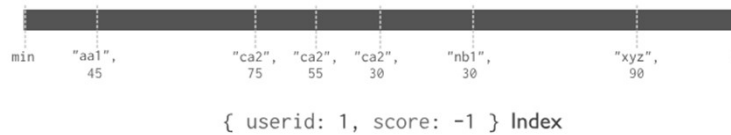
users

168



Índices

- Índices compostos



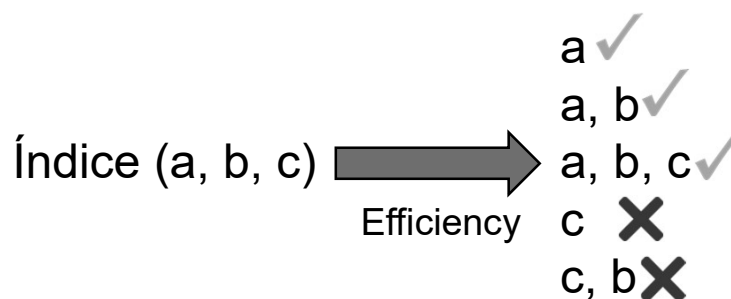
- Note que busca por `score` não utiliza este índice

169



Índices

- Buscas eficientes devem seguir a ordem dos índices



170



Índices

- *“There is no such thing as a free lunch”*
- Leitura (usando índice) serão mais rápidas, mas...
- Alterações em um documento que afetam o índice causam uma reordenação dos índices na memória
 - Escritas geralmente são mais lentas
 - Se você está apenas escrevendo, você não precisa de índices
 - Quando for inserir muitos dados, remova os índices, insira os dados, e só depois crie os índices
 - Provavelmente não desejamos índices em todas as chaves da coleção

171



Índices

- Boa prática
 - Ter um índice por consulta
 - Ter uma consulta por índice

172



Outras Características de SQL

- Comandos de controle de transações
 - Permitem especificar unidades de processamento de BD para fins de concorrência e recuperação
- Construções para especificar privilégios de usuários
 - GRANT e REVOKE
 - Tabelas recebem proprietários
 - Usuários recebem o direito de usar certos comandos SQL sobre a relação
 - Usuários recebem o direito de criar esquemas, tabelas, visões

173



Outras Características de SQL

- Comandos de criação de gatilhos mais complexos
- Propriedades de modelos OO
- Interação com XML

174



Leitura

- Capítulos 4 e 5

175



176